Hardware Modulus Core Using the Restoring Division Algorithm

Final Digital Design / RTL Project — Technical Report

Institution: Faculty of Engineering, Cairo University **Course:** Digital Design / RTL Design **Target Platform:** FPGA (Vivado / ModelSim toolflow) **Data Width:** N = 8 bits (parameterized) **Report Type:** RTL Design and Verification Documentation **Deadline Referenced in Project Proposal:** Thursday, July 23

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Project Objectives
5. Motivation
6. Background
7. Restoring Division Algorithm
8. Hardware Architecture
9. System Specifications
10. Datapath Design
11. Control Unit Design
12. Detailed Module Description
13. FSM Design
14. Datapath Operation
15. Clock-by-Clock Hardware Operation
16. Verification Methodology
17. Testbench Design
18. Simulation Results
19. RTL Viewer
20. Technology Schematic
21. FPGA Synthesis
22. Resource Utilization
23. Timing Analysis
24. Design Limitations
25. Advantages
26. Future Improvements
27. Conclusion
28. References

---

1. Abstract

This report documents the design of an 8-bit hardware Modulus Core implemented in Verilog HDL, targeting FPGA synthesis through the Vivado/ModelSim toolflow. The core computes the remainder of an integer division (`A % B`) using a sequential Restoring Division algorithm rather than the behavioral `%` operator, which synthesizes into large combinational division networks unsuitable for area-constrained designs. The implementation is partitioned into a Datapath (shift register, adder/subtractor, multiplexer) and a Control Unit (a finite state machine with an embedded iteration counter), following a classical two-segment RTL architecture. The report walks

through the algorithm, the RTL structure exactly as implemented in the five provided source files (`modulus_top.v`, `control_unit_fsm.v`, `shift_reg_mod.v`, `add_sub_mod.v`, `mux_2x1_mod.v`), the finite state machine, the intended clock-by-clock operation, and a verification methodology appropriate for the design. Because no testbench, simulation waveform, synthesis report, or timing report was supplied at the time of writing, this report explicitly avoids presenting fabricated results in those categories, and instead explains what each artifact is, why it matters, and how it would be generated. A dedicated **Design Limitations** section documents structural inconsistencies discovered during RTL review — most notably a control signal (`sel`) that is declared but never driven, and named port connections in the top-level instantiation of `shift_reg_mod` that reference ports which do not exist in that module's definition. These are reported as found, without silent correction, in keeping with the goal of describing the implementation as it actually exists.

---

2. Introduction

Division and modulus operations are among the most hardware-expensive arithmetic operations because, unlike addition or multiplication, they cannot be reduced to a fixed, shallow combinational network without significant area cost. When a Verilog designer writes `A % B` directly, most synthesis tools infer a combinational (or IP-core-based) divider that consumes a disproportionate number of LUTs and creates a long, timing-unfriendly critical path. For area-constrained or educational FPGA targets, a **sequential** algorithm that trades latency (multiple clock cycles) for area (small combinational logic reused every cycle) is the standard engineering answer.

This project implements exactly that trade-off: an 8-bit modulus core based on the **Restoring Division algorithm**, executed over multiple clock cycles under the supervision of a finite state machine (FSM). The design is split cleanly into a **Datapath**, which holds and manipulates the numeric registers, and a **Control Unit**, which sequences the datapath's operations and reports completion. This separation of concerns is a foundational RTL design pattern and is reflected directly in the module boundaries of the provided source code.

---

3. Problem Statement

Given two unsigned 8-bit operands — a `dividend` and a `divisor` — the design must compute the arithmetic remainder of `dividend / divisor` using dedicated sequential hardware, without relying on Verilog's behavioral modulus operator, and must do so:

- Deterministically, in a fixed and predictable number of clock cycles,
- Using a datapath composed of a shift register, an adder/subtractor, and a multiplexer (rather than a combinational divider),
- Under the supervision of an explicit finite state machine,
- While detecting the invalid input case of a zero divisor.

---

4. Project Objectives

- Implement a parameterized (`N`-bit, instantiated at `N = 8`) modulus core in synthesizable Verilog.
- Realize the Restoring Division algorithm using discrete RTL building blocks: shift register, adder/subtractor, 2-to-1 multiplexer, and a control FSM.
- Provide a `start`/`done` handshake so the core can be integrated into a larger digital system.
- Detect and flag a divide-by-zero condition (`divisor == 0`) through a dedicated `error` output.
- Keep the datapath and control logic in separate modules to support independent verification and reuse.
- Document the design so that it can be defended and evaluated as a university digital design project.

---

5. Motivation

Behavioral division is one of the few arithmetic operators where the gap between “easy to write” and “efficient to synthesize” is largest. A single line, `remainder = dividend % divisor;`, can synthesize into hundreds of LUTs and a long combinational path if the divisor is not a constant. In contrast, a sequential shift-subtract-restore approach reuses a single N-bit adder/subtractor across N iterations, at the cost of N clock cycles of latency. For an 8-bit operand this is a fixed 8-cycle latency — a very reasonable trade-off for control-plane or low-throughput datapaths (e.g., protocol counters, checksum-related arithmetic, embedded peripherals) where an 8-cycle result is fast enough and area is precious. This motivates the classic “restoring division” hardware technique taught in computer arithmetic courses, and its implementation here as a self-contained, reusable core.

---

6. Background

Restoring division is one of the two classical sequential division algorithms taught alongside non-restoring division. Both operate on a **combined register pair**, conventionally called `A` (accumulator / partial remainder) and `Q` (quotient, initially holding the dividend), which are shifted together as a single wide register. At each iteration:

1. The `AQ` pair is shifted left by one bit.
2. The divisor `M` is subtracted from the upper half `A`.
3. If the result is negative (sign bit / MSB = 1), the subtraction is **undone** (“restored”) by adding `M` back, and the corresponding quotient bit is recorded as `0`.
4. If the result is non-negative, it is kept as the new `A`, and the quotient bit is recorded as `1`.

After `N` iterations (`N` being the operand width), `A` holds the remainder and `Q` holds the quotient. This is precisely the algorithm the provided RTL modules are named and structured after (`add_sub_mod`, `shift_reg_mod`, `mux_2x1_mod`, `control_unit_fsm`), and it is the algorithm this report describes as the design’s target. Section 24 (Design Limitations) discusses where the as-written RTL diverges from a fully correct realization of this textbook algorithm.

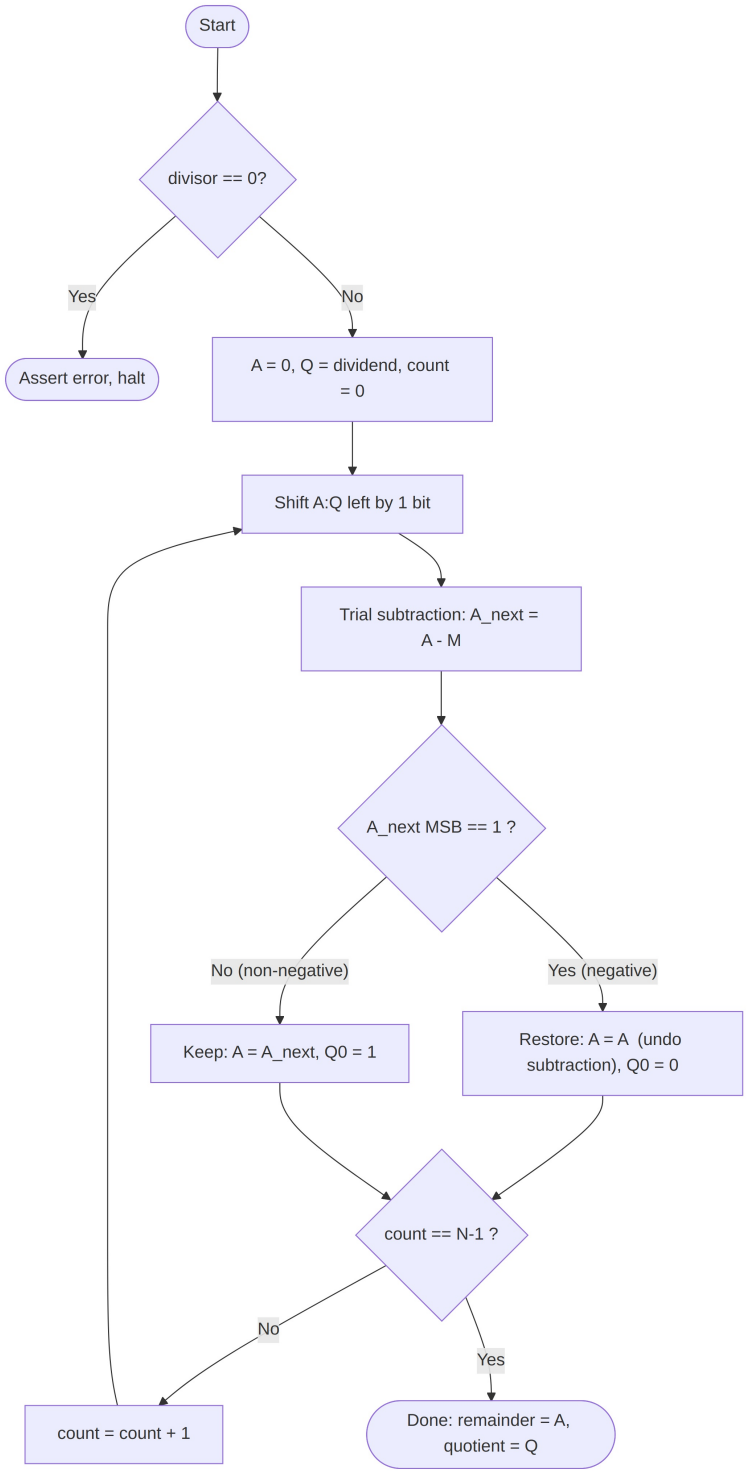
---

7. Restoring Division Algorithm

7.1 Conceptual Algorithm (Target Behavior)

```
A = 0
Q = dividend
for i = 1 to N:
    shift (A,Q) left by 1 bit          // A:Q treated as one 2N-bit
register
    A = A - M                          // trial subtraction
    if A[N-1] == 1:                    // negative result
        A = A + M                      // restore
        Q[0] = 0
    else:
        Q[0] = 1
remainder = A
quotient  = Q
```

7.2 Algorithm Flowchart



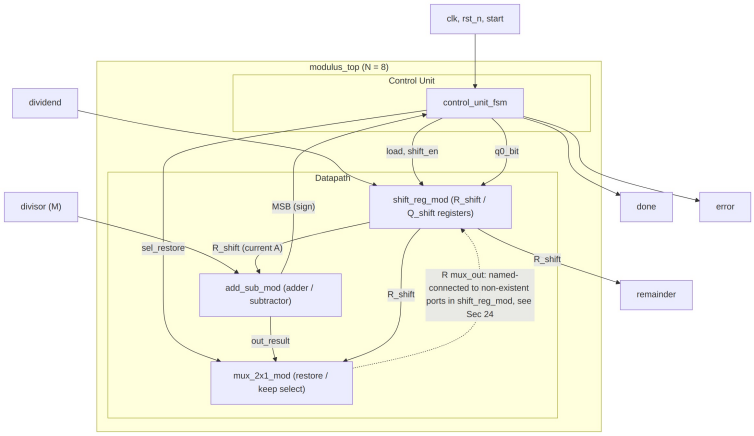
Algorithm Flowchart

This flowchart represents the algorithm the hardware targets. The mapping of each algorithmic step onto the actual FSM states (`IDLE`, `INIT`, `SHIFT`, `CHECK`, `DONE`, `ERROR`) is given in Section 13.

---

8. Hardware Architecture

The RTL is organized as a strict two-segment architecture: a **Datapath** that holds and manipulates data, and a **Control Unit** that generates the control signals driving the datapath, exactly matching the description in the project proposal.



Hardware Architecture

The dashed line marks the feedback path that, per the algorithm, must return the restore/keep decision to the remainder register. As documented in Section 24, this feedback path is wired at the top level to port names (`R`, `Q`) that are not declared in `shift_reg_mod`'s port list.

---

9. System Specifications

Parameter	Value
Operand width (`N`)	8 bits (parameterized)
Clock cycles per operation	8 (one per bit, plus 1 load cycle)
Reset type	Asynchronous, active-low (`rst_n`)
Handshake	`start` (level, held while operating), `done` (level, asserted in `DONE` state)
Error reporting	`error` output, asserted when `divisor == 0` at the time `start` is asserted
Top-level inputs	`clk`, `rst_n`, `start`, `dividend[7:0]`, `divisor[7:0]`
Top-level outputs	`remainder[7:0]`, `done`, `error`
Quotient output	**Not exposed** at the top level (see Section 24)

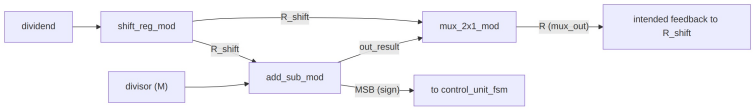
---

10. Datapath Design

The datapath consists of three modules:

- **`shift_reg_mod`** — a combined 16-bit-equivalent shift register realized as two 8-bit registers, `R_shift` (remainder/accumulator, "A") and `Q_shift` (dividend/quotient, "Q"), that shift together on each `shift_en` pulse.
- **`add_sub_mod`** — a purely combinational 8-bit adder/subtractor computing `R - M` or `R + M` depending on a `sel` input, and exposing the result's MSB as a sign flag.

- **`mux_2x1_mod`** — a combinational 2-to-1 multiplexer that selects between the pre-subtraction value of `R` (restore path) and the adder/subtractor's result (keep path), based on a `sel_restore`-derived select line.

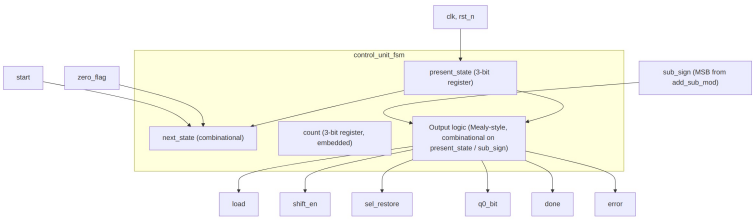


Datapath Design

---

11. Control Unit Design

The Control Unit is realized entirely inside `control_unit_fsm.v`. Unlike the proposal's description of a separate `loop_counter` module, the actual RTL embeds a 3-bit `count` register directly inside the FSM module itself; there is no standalone counter module in the provided source files.



Control Unit Design

---

12. Detailed Module Description

12.1 `modulus_top`

Top-level integration module, parameterized by `N` (default 8). It instantiates `control_unit_fsm`, `shift_reg_mod`, `add_sub_mod`, and `mux_2x1_mod`, and wires them together. It also computes `w_zero_flag = (divisor == 0)` combinationally and forwards it to the FSM, and assigns the top-level `remainder` output directly from the shift register's `R_shift` output (`w_r_shift`). There is no `quotient` output port, and the internally computed `w_sub_sel` wire (intended to drive `add_sub_mod`'s `sel` input) is declared but never assigned anywhere in the module (see Section 24).

12.2 `control_unit_fsm`

A synchronous Mealy-style FSM with six states (`IDLE`, `INIT`, `SHIFT`, `CHECK`, `DONE`, `ERROR`), a 3-bit `present_state`/`next_state` pair, and an internal 3-bit `count` register used to track the 8 required shift/check iterations. It drives five control outputs (`load`, `shift_en`, `sel_restore`, `q0_bit`, `done`) plus an `error` flag, based on `start`, `sub_sign` (the sign bit from the adder/subtractor), and `zero_flag` (divide-by-zero indicator).

12.3 `shift_reg_mod`

Holds the two working registers `R_shift` (remainder/accumulator) and `Q_shift` (dividend → quotient). On active-low asynchronous reset both registers clear to zero. On `load`, `Q_shift` is initialized with the `divide_in` (dividend) value and `R_shift` is cleared. On `shift_en`, both

registers shift left together as one logical unit: `R_shift` receives its own upper bits shifted up with the incoming bit being `Q_shift`'s current MSB, and `Q_shift` receives its own upper bits shifted up with the incoming bit being the external `q_in` (the FSM's `q0_bit`). The module's port list contains **only** `clk, rst_n, divide_in, shift_en, q_in, load, R_shift, Q_shift` — it has no port through which an external "restored/kept" remainder value can be written into `R_shift` (see Section 24).

12.4 `add_sub_mod`

A combinational block computing `out_result = R - M` when `sel = 1`, or `out_result = R + M` when `sel = 0`, and exposing `MSB = out_result[N-1]` as the sign/borrow indicator used by the control unit to decide restore vs. keep.

12.5 `mux_2x1_mod`

A combinational 2-to-1 multiplexer: `R = s ? r_res : r_no_res`. In the top-level instantiation, `r_res` is connected to the pre-subtraction value of `R_shift` (the "restore" choice) and `r_no_res` is connected to the adder/subtractor's `out_result` (the "keep the subtraction" choice), selected by `sel_restore` (`s`).

---

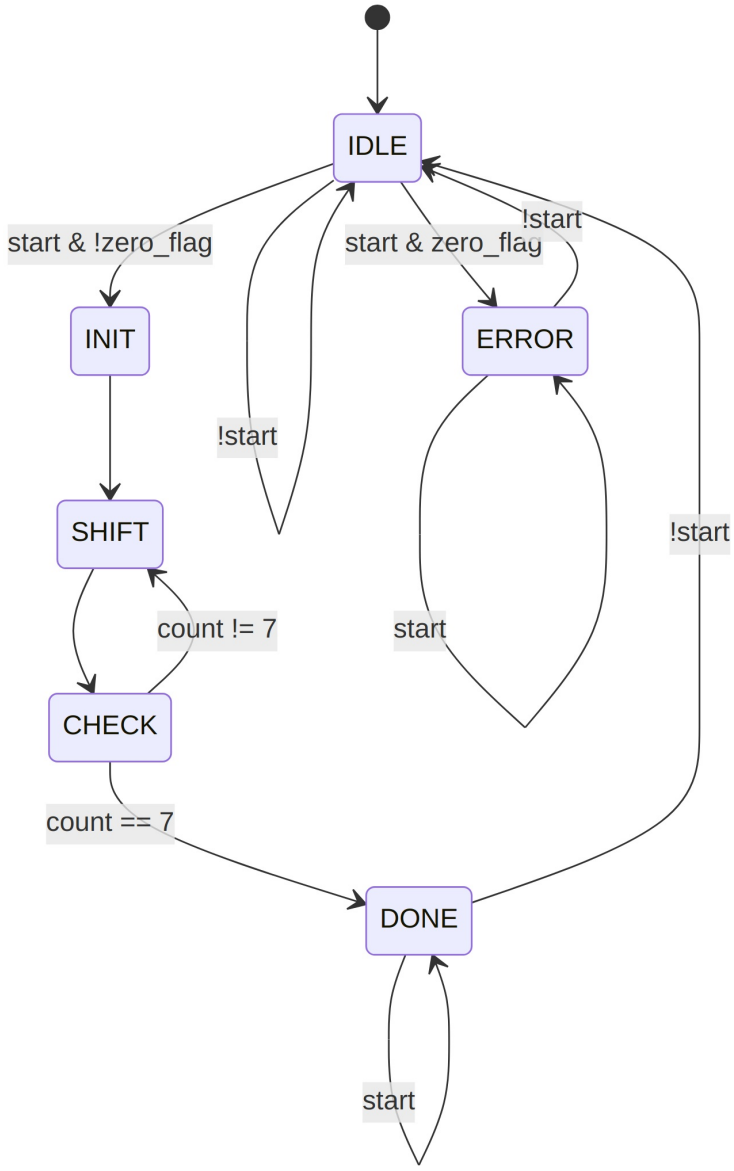
13. FSM Design

13.1 States

State	Encoding	Purpose
`IDLE`	3'b000	Waits for `start`. All control outputs held low.
`INIT`	3'b001	Asserts `load`, causing `Q_shift ← dividend` and `R_shift ← 0`. Also resets `count` to 0.
`SHIFT`	3'b010	Asserts `shift_en`, causing `R_shift`/`Q_shift` to shift left by one bit.
`CHECK`	3'b011	Combinationally evaluates `sub_sign` (the sign of the trial subtraction) and sets `sel_restore` and `q0_bit` accordingly. Increments `count`.
`DONE`	3'b100	Asserts `done`. Holds in `DONE` while `start` remains asserted; returns to `IDLE` once `start` is deasserted.
`ERROR`	3'b101	Asserted when `start` is applied with `divisor == 0`. Asserts `error`. Holds while `start` remains

	asserted; returns to `IDLE` once `start` is deasserted.

13.2 State Diagram



FSM State Diagram

13.3 Transition Logic

- **`IDLE → INIT`**: taken when `start = 1` and `zero_flag = 0`.
- **`IDLE → ERROR`**: taken when `start = 1` and `zero_flag = 1` (invalid divisor).
- **`INIT → SHIFT`**: unconditional, single-cycle pass-through used purely to issue the `load` pulse.
- **`SHIFT → CHECK`**: unconditional, single-cycle pass-through used purely to issue the `shift_en` pulse.
- **`CHECK → SHIFT`**: taken while `count != 7`, i.e., while fewer than 8 shift/check iterations have completed.
- **`CHECK → DONE`**: taken when `count == 7`, i.e., after the 8th iteration.

- **`DONE → IDLE` / `DONE → DONE`**: the FSM holds in `DONE` (with `done` asserted) as long as `start` remains high, and only returns to `IDLE` once the caller deasserts `start` — a standard level-sensitive handshake so the requester has time to sample `done`/`remainder`.
- **`ERROR → IDLE` / `ERROR → ERROR`**: symmetric behavior to `DONE`, for the invalid-divisor case.

13.4 Control Signals

Signal	Asserted in	Effect
`load`	`INIT`	Loads `dividend` into `Q_shift`, clears `R_shift`
`shift_en`	`SHIFT`	Shifts `R_shift`/`Q_shift` left by one bit
`sel_restore`	`CHECK`, when `sub_sign = 1`	Selects the "restore" (pre-subtraction) path through the mux
`q0_bit`	`CHECK`, set to `1` when `sub_sign = 0`, else `0`	Value shifted into `Q_shift`'s LSB on the *next* `SHIFT`
`done`	`DONE`	Signals completion; `remainder` is valid
`error`	`ERROR`	Signals invalid input (`divisor == 0`)

14. Datapath Operation

14.1 Registers

- **`R_shift` (A / remainder accumulator, 8 bits):** Cleared on reset or `load`; shifts left on `shift_en`, bringing in `Q_shift`'s current MSB. This is the register that, per the algorithm, must also absorb the subtract/restore decision — see Section 24 for the discrepancy found here.
- **`Q_shift` (Q / dividend-then-quotient, 8 bits):** Loaded with `dividend` on `load`; shifts left on `shift_en`, bringing in the FSM's `q0_bit` (computed in the *previous* `CHECK` state) as the new LSB. Because `q0_bit` is set combinationally in `CHECK` and only takes effect in the following `SHIFT`, the quotient-bit accumulation timing is internally consistent within `shift_reg_mod` itself.

14.2 Shifting

Both registers shift **together**, each independently in `shift_reg_mod`:

```
R_shift <= {R_shift[N-2:0], Q_shift[N-1]}
Q_shift <= {Q_shift[N-2:0], q_in}
```

This realizes the conceptual single wide `A:Q` shift register as two cooperating 8-bit registers, with `Q_shift`'s outgoing MSB feeding `R_shift`'s incoming LSB — exactly the classical restoring-division shift step.

14.3 Subtraction

`add_sub_mod` continuously computes `R_shift - divisor` (when `sel = 1`) and exposes the sign of that result as `MSB`. This value is available to the control unit as `sub_sign` on every clock, and is sampled by the FSM during `CHECK`.

14.4 Restoring

When `sub_sign = 1` (the trial subtraction produced a negative result), the control unit asserts `sel_restore`, which drives `mux_2x1_mod` to select the pre-subtraction value of `R_shift` — i.e., to discard the subtraction and “restore” the prior accumulator value, per the algorithm.

14.5 Quotient Generation

Each `CHECK` cycle sets `q0_bit` to `1` when the subtraction was non-negative (`sub_sign = 0`) and `0` when it was negative, ready to be shifted into `Q_shift` on the following `SHIFT` pulse.